

Proyecto 1

Uso de un protocolo existente — Model Context Protocol

Entrega parcial. Este documento cubre la primera parte del proyecto: los incisos 4 y 5 (servidores MCP locales), junto con los incisos 8 y 10 del reporte aplicados a esa primera parte.

Repositorio: github.com/UVG-Work/Proyecto-1-Chat-MCP-

Implementación: TypeScript sobre Node.js, sin SDK de MCP

Revisión del protocolo: 2025-11-25

1. Alcance de esta entrega

El proyecto consiste en un chatbot que actúa como anfitrión (*host*) de MCP y utiliza las herramientas expuestas por varios servidores. Esta entrega parcial documenta únicamente la primera parte: el uso de los servidores oficiales de Anthropic y la construcción de un servidor MCP propio basado en un caso de uso de industria.

INCISO	CONTENIDO	SECCIÓN
4	Uso de los servidores MCP locales oficiales: Filesystem y Git	Sección 3
5	Servidor MCP propio, ejecutado localmente, basado en un caso de uso de industria	Sección 4
8	Especificación del servidor desarrollado: parámetros, endpoints, métodos	Sección 5
10	Conclusiones y comentarios sobre esta parte del proyecto	Sección 6

Quedan fuera de este documento los incisos 6 y 7 (servidor remoto y análisis con Wireshark), que corresponden a la segunda parte, así como el inciso 9, que depende del análisis de captura de esa segunda parte.

2. Restricción de implementación

El enunciado (sección 3.1) prohíbe el uso de librerías o SDKs que implementen MCP, tales como FastMCP. En consecuencia, en este proyecto se escribieron a mano:

- el formato y el intercambio de mensajes JSON-RPC 2.0;
- la fase de inicialización (`initialize` y `notifications/initialized`);
- la negociación de versión de protocolo y de capacidades;
- el transporte stdio, tanto del lado del cliente como del servidor.

Es importante señalar que esto aplica al *cliente* además del servidor. Para cumplir el inciso 4 el cliente escrito a mano debe comunicarse con servidores desarrollados por terceros, lo cual obliga a respetar la especificación con exactitud; cualquier desviación en el formato de trama o en el ciclo de vida impide la conexión.

Alcance de la restricción. La prohibición se refiere a implementaciones de MCP. El paquete `openai` se utiliza únicamente como cliente HTTP hacia la API de OpenRouter para conversar con el modelo de lenguaje, que no forma parte del protocolo MCP. Todo byte de MCP en este proyecto es producido por código propio en `src/mcp/` y `src/server/`.

3. Inciso 4: servidores MCP locales oficiales

El anfitrión se conecta a los dos servidores oficiales de referencia mediante el transporte stdio: el cliente lanza cada servidor como un subproceso e intercambia mensajes JSON-RPC delimitados por saltos de línea a través de su entrada y salida estándar.

SERVIDOR	IMPLEMENTACIÓN	INVOCACIÓN	HERRAMIENTAS
Filesystem	secure-file-system-server 0.2.0 (TypeScript)	<code>npx -y @modelcontextprotocol/server-file-system ./sandbox</code>	14
Git	mcp-git 1.29.0 (Python)	<code>uvx mcp-server-git</code>	12

Ambos negocian la revisión `2025-11-25` y declaran la capacidad `tools`. Conviene notar que el servidor de Git es un paquete de Python y no de Node, por lo que el proyecto requiere `uv` instalado aunque el resto esté escrito en TypeScript.

3.1 Escenario demostrado

Se solicita al chatbot, en lenguaje natural, que cree un archivo README, lo agregue al repositorio y realice un commit. El modelo decide por sí mismo qué herramientas usar y en qué orden; el anfitrión únicamente traduce esas decisiones a llamadas `tools/call` y las enruta al servidor correspondiente.

Instrucción: "Inside the demo-repo directory, create a file named README.md describing this project. Then add it to the git repository in that same directory and commit it with a descriptive message. Finally show me the git log."

<code>filesystem__list_allowed_directories</code>	<code>{}</code>	<code>ok</code>
<code>filesystem__list_directory</code>	<code>{"path": "../sandbox"}</code>	<code>ok</code>
<code>filesystem__write_file</code>	<code>{"path": "../README.md"}</code>	<code>ok</code>
<code>git__git_status</code>	<code>{"repo_path": "..."} </code>	<code>ok</code>
<code>git__git_add</code>	<code>{"files": ["README.md"]}</code>	<code>ok</code>
<code>git__git_commit</code>	<code>{"message": "Add README..."}</code>	<code>ok</code>
<code>git__git_log</code>	<code>{"repo_path": "..."} </code>	<code>ok</code>

Resultado verificado en disco:
`dc5c5e7 Add README.md describing the MCP chat host for networking course`
`README.md | 22 ++++++`

El commit existe realmente en el repositorio de prueba y fue creado por el chatbot a través de MCP, no por la línea de comandos. Se observa además que el modelo combina dos servidores distintos en una sola tarea: escribe el archivo con el servidor de Filesystem y lo versiona con el de Git.

3.2 Limitación encontrada

El servidor oficial de Git no expone `git_init`. La versión 1.29.0 ofrece doce herramientas (`git_status`, `git_diff`, `git_commit`, `git_add`, `git_reset`, `git_log`, `git_create_branch`, `git_checkout`, `git_show`, `git_branch` y las variantes de diff), ninguna de las cuales crea un repositorio. Por lo tanto la creación del repositorio se realiza mediante un script de preparación (`npm run setup:demo`), mientras que la creación del README, su incorporación al índice y el commit sí se ejecutan por medio de MCP. Se documenta la limitación en lugar de ocultarla.

3.3 Registro de interacciones

Todas las solicitudes y respuestas con los servidores MCP quedan registradas. En el cliente de terminal, el comando `/log` muestra cada trama con su marca de tiempo, dirección, servidor y clasificación; `/log full` incluye la carga JSON completa. El registro se toma en la capa de transporte, de modo que resulta completo por construcción y no por recordar invocarlo en cada punto del código.

4. Inciso 5: servidor MCP propio

4.1 Caso de uso de industria

El servidor implementa la **mesa de servicio de primer nivel de un proveedor de internet (ISP)**. Es un caso de uso real y directamente relacionado con la materia: cuando un abonado llama reportando lentitud, el agente debe identificar la cuenta, consultar la telemetría del circuito, descartar una avería de zona conocida y decidir si la evidencia justifica enviar un técnico.

Ese encadenamiento es lo que hace el caso apropiado para MCP: el modelo no invoca una sola herramienta, sino que ordena cinco y decide en cada paso qué hacer con el resultado anterior. Cada una de esas herramientas correspondería, en un operador real, a un sistema distinto.

Todos los datos son sintéticos y no contienen información de abonados reales. Los diagnósticos se basan en umbrales determinísticos y no en valores aleatorios, de modo que una demostración produce siempre el mismo resultado.

4.2 Herramientas expuestas

HERRAMIENTA	PROPÓSITO
<code>lookup_subscriber</code>	Localiza una cuenta por nombre, identificador, circuito o teléfono. Devuelve el identificador de circuito que requieren las demás herramientas.
<code>get_link_metrics</code>	Telemetría del enlace: latencia, jitter, pérdida de paquetes, relación señal a ruido, throughput medido, tiempo en servicio y número de caídas.
<code>run_link_diagnostics</code>	Evalúa la telemetría contra los umbrales del operador y devuelve un diagnóstico con la acción recomendada.
<code>check_zone_outage</code>	Incidentes conocidos que afectan una zona de servicio, con causa, estado y tiempo estimado de resolución.
<code>open_incident_ticket</code>	Escala el caso creando un ticket de incidente. Es la única herramienta que modifica estado.

4.3 Escenario demostrado

Instrucción: "La clienta Maria Elena Ramirez llama porque su internet esta muy lento. Investiga que ocurre con su servicio y, si corresponde, abre un ticket."

<code>lookup_subscriber</code>	<code>{"query":"Maria Elena Ramirez"}</code>	<code>ok</code>
<code>get_link_metrics</code>	<code>{"circuit_id":"GT-CIR-004821"}</code>	<code>ok</code>
<code>check_zone_outage</code>	<code>{"zone":"ZONA-MIXCO-03"}</code>	<code>ok</code>
<code>run_link_diagnostics</code>	<code>{"circuit_id":"GT-CIR-004821"}</code>	<code>ok</code>
<code>open_incident_ticket</code>	<code>{"subscriber_id":"SUB-100482", ...}</code>	<code>ok</code>

Hallazgos: pérdida de paquetes 8.2 % (umbral crítico 5 %), SNR 14.1 dB (umbral crítico 15 dB), jitter 19.4 ms, 11 caídas en 24 h, throughput 62 de 300 Mbps. Sin incidencias de zona que expliquen el síntoma.

Diagnóstico: falla crítica del enlace. Ticket TCK-4401 abierto, severidad alta, asignado a Field Operations.

El modelo secuencia las cinco llamadas por su cuenta a partir de la descripción del problema y de las descripciones de las herramientas; no existe ninguna lógica en el anfitrión que codifique este orden.

4.4 Datos de prueba

El conjunto de datos está construido de modo que cada abonado ejercita una ruta diagnóstica distinta, permitiendo demostrar los cuatro diagnósticos posibles.

ABONADO	CIRCUITO	CONDICIÓN
SUB-100482	GT-CIR-004821	Degradado: 8.2 % de pérdida, SNR 14.1 dB, 11 caídas
SUB-100731	GT-CIR-005190	Sano: todas las métricas dentro de umbral
SUB-100955	GT-CIR-005544	Caído por avería de zona activa (corte de fibra)
SUB-101204	GT-CIR-006012	Enlace sano pero cuenta suspendida
SUB-101488	GT-CIR-006377	Congestión: 137 ms de latencia y 42.6 ms de jitter sin pérdida

Los dos últimos casos son deliberados: una cuenta suspendida y un corte de fibra se parecen, vistos sólo desde la telemetría, a un enlace averiado. Enviar un técnico en cualquiera de los dos casos sería un desplazamiento inútil, por lo que el diagnóstico los distingue de forma explícita.

5. Inciso 8: especificación del servidor desarrollado

Nombre del servidor	<code>noc-support-desk</code>
Versión	1.0.0
Revisión de MCP	<code>2025-11-25</code> , con negociación hacia atrás a 2025-06-18, 2025-03-26 y 2024-11-05
Capacidades declaradas	<code>tools</code> con <code>listChanged: false</code>
Transporte (esta entrega)	stdio
Punto de entrada	<code>src/server/stdio-main.ts</code> , ejecutable con <code>npm run noc:stdio</code>

5.1 Métodos soportados

MÉTODO	TIPO	COMPORTAMIENTO
<code>initialize</code>	solicitud	Devuelve versión negociada, capacidades, identificación del servidor e instrucciones de uso.
<code>notifications/initialized</code>	notificación	Se consume y nunca se responde, conforme a JSON-RPC.
<code>ping</code>	solicitud	Devuelve un resultado vacío.
<code>tools/list</code>	solicitud	Devuelve las cinco herramientas con su esquema de entrada.
<code>tools/call</code>	solicitud	Ejecuta una herramienta y devuelve su resultado.

Cualquier otro método devuelve el error JSON-RPC `-32601` (método no encontrado).

5.2 Parámetros de las herramientas

HERRAMIENTA	PARÁMETRO	TIPO	DESCRIPCIÓN
<code>lookup_subscriber</code>	<code>query</code>	string	Nombre, <code>SUB-#####</code> , <code>GT-CIR-#####</code> o teléfono. Requerido.
<code>get_link_metrics</code>	<code>circuit_id</code>	string	Identificador de circuito. Requerido.
<code>run_link_diagnostics</code>	<code>circuit_id</code>	string	Identificador de circuito. Requerido.
<code>check_zone_outage</code>	<code>zone</code>	string	Zona de servicio, por ejemplo <code>ZONA-GUATE-07</code> . Requerido.
<code>open_incident_ticket</code>	<code>subscriber_id</code>	string	Identificador exacto del abonado. Requerido.
	<code>summary</code>	string	Descripción del fallo y su evidencia. Requerido.
	<code>severity</code>	string	Uno de <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code> . Requerido.

5.3 Umbrales de diagnóstico

MÉTRICA	DEGRADADO	CRÍTICO
Pérdida de paquetes	$\geq 1 \%$	$\geq 5 \%$
Latencia	$\geq 60 \text{ ms}$	$\geq 120 \text{ ms}$
Jitter	$\geq 15 \text{ ms}$	$\geq 30 \text{ ms}$
Relación señal a ruido	$\leq 20 \text{ dB}$	$\leq 15 \text{ dB}$
Caídas en 24 horas	≥ 5	—

Los diagnósticos posibles son:

DIAGNÓSTICO	SIGNIFICADO Y ACCIÓN RECOMENDADA
<code>healthy</code>	Todo dentro de umbral; revisar el equipo del cliente antes de escalar.
<code>link-fault</code>	Uno o más umbrales excedidos; abrir ticket con la severidad correspondiente.
<code>zone-outage</code>	Una avería conocida explica el síntoma; informar el tiempo estimado y no duplicar el ticket.
<code>account-suspended</code>	El enlace está sano pero la cuenta está suspendida; derivar a facturación.

5.4 Endpoints y transporte

En esta primera parte el servidor se ejecuta localmente sobre stdio, por lo que no expone endpoints de red. Las reglas de trama son las que fija la especificación:

- un mensaje JSON-RPC por línea, codificado en UTF-8;
- los mensajes no pueden contener saltos de línea incrustados;
- la salida estándar transporta únicamente mensajes MCP, por lo que todo diagnóstico del servidor se escribe en la salida de error;
- el cliente no debe interpretar la salida de error como una condición de fallo;
- cerrar la entrada estándar termina el servidor de forma ordenada.

El mismo servidor está preparado para ejecutarse sobre el transporte Streamable HTTP, en cuyo caso sí expone un endpoint `/mcp`. Eso corresponde al inciso 6 y se documenta en la entrega final.

5.5 Manejo de errores

CÓDIGO	SIGNIFICADO	CUÁNDO SE PRODUCE
<code>-32700</code>	Error de análisis	La carga no es JSON válido.
<code>-32600</code>	Solicitud inválida	No es un mensaje JSON-RPC 2.0 válido.
<code>-32601</code>	Método no encontrado	Método desconocido.
<code>-32602</code>	Parámetros inválidos	Herramienta inexistente, argumento requerido ausente, tipo incorrecto o valor fuera del conjunto permitido.
<code>-32603</code>	Error interno	Fallo inesperado del servidor.

Conviene distinguir dos situaciones que suelen confundirse. Una herramienta que se ejecuta y *falla* no produce un error de JSON-RPC: devuelve una respuesta exitosa cuyo resultado lleva `isError: true` junto con un texto explicativo, de modo que el modelo pueda leer el fallo y reaccionar. Los códigos anteriores se reservan para errores de protocolo.

5.6 Ejemplo de intercambio

```
--> {"jsonrpc":"2.0","id":1,"method":"initialize",
    "params":{"protocolVersion":"2025-11-25","capabilities":{},
              "clientInfo":{"name":"uvg-mcp-chat-host","version":"1.0.0"}}}

<-- {"jsonrpc":"2.0","id":1,"result":{"protocolVersion":"2025-11-25",
    "capabilities":{"tools":{"listChanged":false}},
    "serverInfo":{"name":"noc-support-desk","version":"1.0.0"},
    "instructions":"This server exposes the first-line support desk..."}}

--> {"jsonrpc":"2.0","method":"notifications/initialized"}

--> {"jsonrpc":"2.0","id":2,"method":"tools/list","params":{}}
<-- {"jsonrpc":"2.0","id":2,"result":{"tools":[ ... 5 herramientas ... ]}}

--> {"jsonrpc":"2.0","id":3,"method":"tools/call",
    "params":{"name":"lookup_subscriber",
              "arguments":{"query":"Maria Elena Ramirez"}}}
<-- {"jsonrpc":"2.0","id":3,"result":{"content":[{"type":"text",
    "text":"Found 1 matching subscriber(s): SUB-100482 ..."}],
    "structuredContent":{"matches":[ ... ]}}}
```

El campo `instructions` merece una nota: el servidor describe en él el orden recomendado de uso de sus herramientas, y el anfitrión lo reenvía al modelo. De este modo el servidor documenta su propio uso a través del protocolo, en lugar de que el anfitrión codifique conocimiento sobre un servidor en particular.

6. Inciso 10: conclusiones y comentarios

6.1 Sobre el protocolo

MCP es una convención, no un formato de trama. No hay una cabecera de MCP ni un puerto de MCP. Lo que existe es JSON-RPC 2.0 con un conjunto acordado de nombres de método, un ciclo de vida y una negociación de capacidades. Esa es precisamente la razón por la que es viable implementarlo a mano, y también por la que el mismo código de servidor puede atender transportes distintos sin modificarse.

La interoperabilidad es el objetivo del protocolo y se comprueba al conectarse con servidores ajenos. Escribir un servidor propio es cómodo porque uno responde a los mensajes que uno mismo eligió. El inciso 4 obliga a lo contrario: el cliente escrito a mano debe hablar con implementaciones de terceros, y ahí es donde cada ambigüedad de la especificación se convierte en un fallo concreto.

6.2 Dificultades encontradas

La negociación de versión no puede fijarse. La intuición inicial es enviar la revisión más reciente y esperar la misma de vuelta. La especificación establece que el servidor responde con *su* revisión preferida y que el cliente debe aceptarla si la soporta, o desconectarse en caso contrario. Un cliente con una única versión fija funciona contra el servidor propio y falla contra los oficiales: el peor tipo de error, porque las pruebas locales nunca lo revelan.

La salida estándar es sagrada en stdio. Un servidor no debe escribir en la salida estándar nada que no sea un mensaje MCP. Un simple mensaje de depuración corrompe la trama y rompe la sesión completa. Por eso todos los diagnósticos del servidor propio se escriben en la salida de error, y el cliente trata esa salida como informativa y nunca como una señal de fallo.

Las tramas deben reensamblarse, no suponerse. Un solo evento de lectura sobre el flujo puede contener medio mensaje, varios mensajes, o ambas cosas. El transporte acumula lo recibido y divide en los saltos de línea, conservando el resto para la siguiente lectura.

Lanzar los servidores oficiales en Windows tiene su detalle. `npx` y `uvx` son archivos `.cmd`, que Node no puede ejecutar directamente desde una restricción de seguridad introducida a raíz de CVE-2024-27980. Es necesario pasar por el intérprete de comandos y encargarse del entrecomillado de los argumentos de forma explícita.

Una herramienta ausente obliga a documentar en vez de disimular. El caso de `git_init`, descrito en la sección 3.2, es el ejemplo: el enunciado sugiere que el chatbot cree el repositorio, pero el servidor oficial actual no ofrece esa operación. Se optó por dejar la creación fuera de MCP, indicarlo con claridad y mantener dentro del protocolo todo lo que sí es posible.

6.3 Lecciones aprendidas

El diseño del transporte como interfaz paga desde el primer día. Separar el transporte del cliente y del núcleo del servidor permitió que el mismo objeto de servidor atiende stdio y HTTP sin duplicar lógica. Si esa separación se hubiera pospuesto, la segunda parte del proyecto habría implicado reescribir el servidor en lugar de añadir un punto de entrada.

Un solo punto de registro produce un registro completo. Al capturar cada trama en la capa de transporte, el requisito de mantener y mostrar un log de todas las interacciones se satisface por construcción, y no depende de recordar registrar en cada lugar del código.

Un buen caso de uso para un agente es aquel que requiere encadenar decisiones. Una herramienta aislada no demuestra gran cosa. El valor de MCP se aprecia cuando el modelo debe elegir entre varias herramientas, usar el resultado de una para construir la llamada siguiente y decidir cuándo detenerse. El escenario de la mesa de servicio se diseñó expresamente con esa forma.

La descripción de una herramienta es parte de su interfaz. El modelo elige qué llamar leyendo el nombre, la descripción y el esquema de entrada. Una descripción vaga produce llamadas incorrectas con la misma facilidad que un error de programación, de manera que redactarlas con cuidado es trabajo de ingeniería y no de documentación.

6.4 Comentario final sobre esta parte

La primera parte del proyecto deja el protocolo implementado y verificado en las dos direcciones que importan: un cliente propio que se entiende con servidores desarrollados por terceros, y un servidor propio que se entiende con ese mismo cliente. Sobre esa base, la segunda parte consiste en cambiar el transporte y observar el resultado en la red, sin modificar ni la lógica de las herramientas ni el anfitrión.

Documento correspondiente a la entrega parcial. Incisos 4, 5, 8 y 10 de la primera parte. El código, la especificación completa y el registro de versiones se encuentran en el repositorio del proyecto.